Table of Contents

QUESTION 1 COMMENTING

DO NOT REMOVE THE LINE BELOW

```
clear; close all;
```

==========

==========

1. COMMENTING

==========

==========

MAKE SURE FILE BELOW IS IN SAME DIRECTORY AS THIS FILE

```
type('eel3135_lab03_comment.m')
```

```
% USER DEFINED VARIABLES
w = 20;            % Width
x = 0:1:79;        % Horiztonal Axis
y = 0:1:79;        % Vertical Axis

% Create a simple "bright spot" (an oval-shaped blob) in the image near
(x=20, y=50);
% then rounding values so those close to the center become 1, and values far
away become 0.
z = round(exp(-1/w.^2*(((y.'-50)/1.5).^2+((x-20)).^2)));

% Send the image through two processing steps: first resize it (System 1),
```

```
% then shift and dim it to produce the final image (System 2).
[xs,ys,zs] = image_system1(z,3,6);
za         = image_system2(zs,90,-4);

% PLOT RESULT WITH SUBPLOT
figure(1);
subplot(1,3,1);             % Create a 1×3 subplot layout and select the
first plot.
imagesc(x, y, z);           % Display the original image with color-scaled
intensity values.
axis square; axis image;    % Keep the image from being stretched or
distorted.
title('Original')
subplot(1,3,2);             % Select the second subplot for the System 1
output.
imagesc(xs, ys, zs);        % Display the image after processing by System 1.
axis square; axis image;    % Maintain correct image aspect ratio.
title('After System 1')
subplot(1,3,3);             % Select the third subplot for the System 2
output.
imagesc(xs, ys, za);        % Display the image after processing by System 2.
axis square; axis image;    % Maintain correct image aspect ratio.
title('After System 2')


function [xs, ys, zs] = image_system1(z,Ux,Dy)
%IMAGE_SYSTEM1   Resize an image by upsampling horizontally by Ux
%               and downsampling vertically by Dy.

% Preallocate the output image after vertical downsampling and horizontal
upsampling.
zs = zeros(ceil(size(z,2)/Dy),ceil(Ux*size(z,1)));

% Create new x- and y-axis values that match the resized image dimensions.
ys = 1:ceil(size(z,1)/Dy);
xs = 1:ceil(Ux*size(z,2));

% Copy every Dy-th row of the original image and place columns every Ux
positions,
% leaving zeros in between to stretch the image horizontally.
zs(1:end,1:Ux:end) = z(1:Dy:end,1:end);

end

function [za] = image_system2(z,Sx,Sy)
%IMAGE_SYSTEM2   Shift an image by (Sx, Sy) and reduce its intensity by half.

% Initialize the output image with zeros (same size as input).
za = zeros(size(z,1), size(z,2));

for nn = 1:size(z,1)
    for mm = 1:size(z,2)
        % Check that the shifted pixel location is within the image
boundaries.
```

```
        if nn > Sy && nn-Sy < size(z,1) && mm > Sx && mm-Sx < size(z,2)
            % Copy the shifted pixel into the output image and scale its
value.
            za(nn,mm) = 1/2*z(nn-Sy,mm-Sx);
        end
    end
end


end
```

==========

==========

2. SAMPLING IN IMAGES

==========

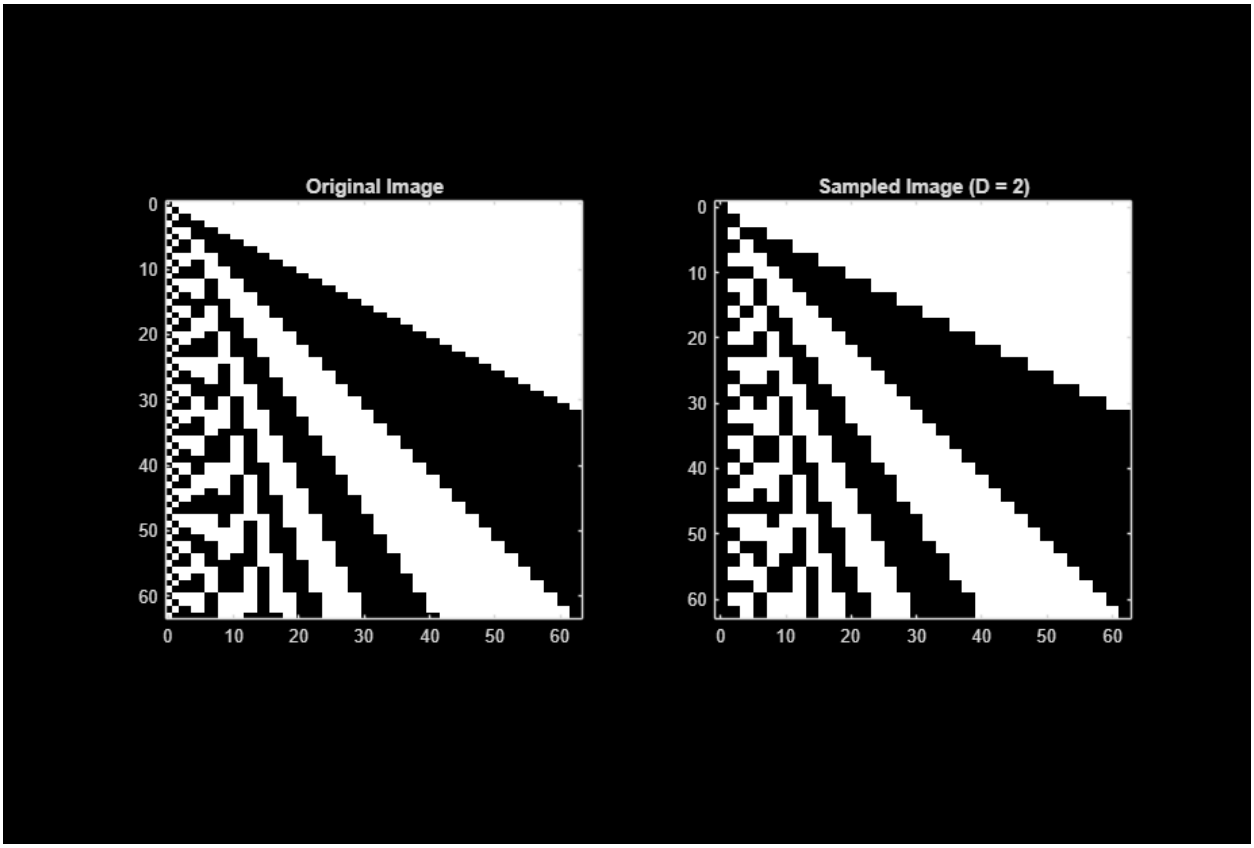
==========

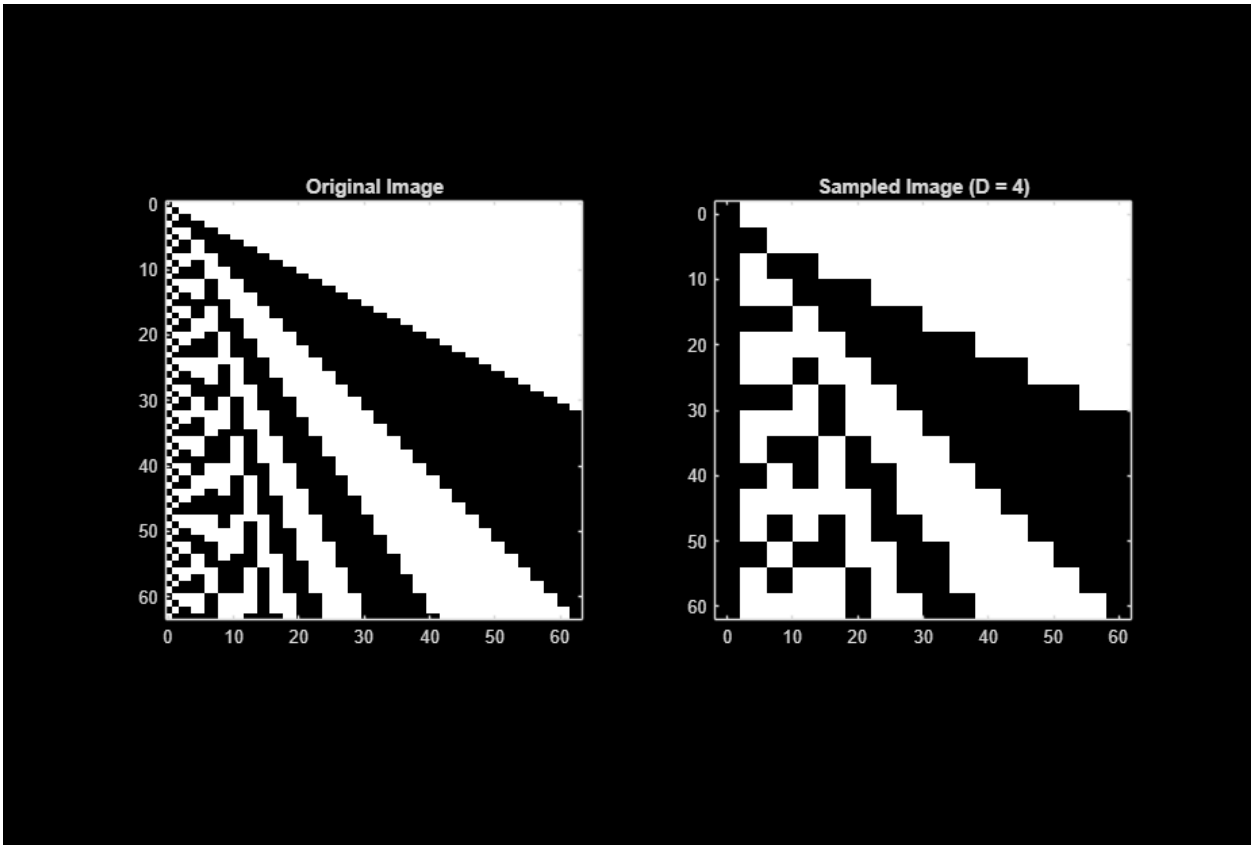
```
% MAKE IMAGE
N = 64;      % image size
x = 0:N-1;   % x-axis
y = 0:N-1;   % y-axis
[xgrid,ygrid] = meshgrid(x,y);
b_min = 1; b_max = 32;
b = round( b_min + (b_max-b_min) * (xgrid/(N-1)) );    % spatially varying
block size
z = mod( floor(xgrid./b) + floor(ygrid./b), 2 );       % final image


% ------------------------------------------------------------------
% 2(a)
% ------------------------------------------------------------------
% Only need to modify function -- this area can be empty


% ------------------------------------------------------------------
% 2(b)
% ------------------------------------------------------------------

D = 2;
[xs2, ys2, zs2] = sample(z, D);

figure;
colormap(gray)

subplot(1,2,1);               % Left plot
imagesc(x, y, z);             % Original image
axis image;
```

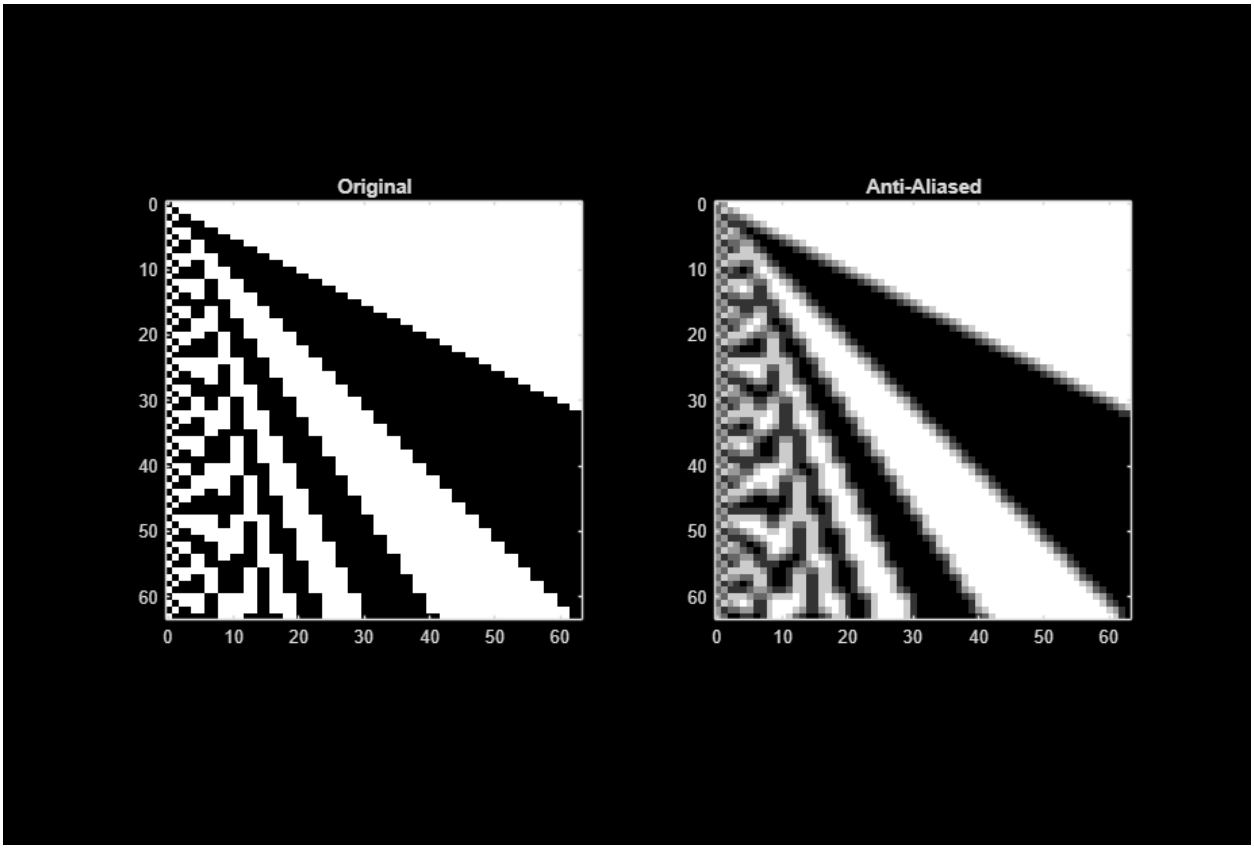
```
title('Original Image');

subplot(1,2,2);              % Right plot
imagesc(xs2, ys2, zs2);      % Sampled image (D = 2)
axis image;
title('Sampled Image (D = 2)');



% ------------------------------------------------------------------
% 2(c)
% ------------------------------------------------------------------
D = 4;
[xs4, ys4, zs4] = sample(z, D);

figure;
colormap(gray)

subplot(1,2,1);              % Left plot
imagesc(x, y, z);            % Original image
axis image;
title('Original Image');

subplot(1,2,2);              % Right plot
imagesc(xs4, ys4, zs4);      % Sampled image (D = 4)
axis image;
title('Sampled Image (D = 4)');
% ------------------------------------------------------------------
% 2(d)
% ------------------------------------------------------------------
% ANSWER QUESTION BELOW
% After applying sampling in some cases, specific regions with rapid pixel-
to-pixel changes (such as the small checker-like
% patterns near the left side) appear distorted or change shape. These
regions contain
% high spatial frequencies, which are not captured well when pixels are
skipped.
% Smoother regions with slow intensity changes (the large diagonal areas)
are less affected
% because they contain mostly low spatial frequencies. The regions with the
% smaller alternating pixels, like along the left wall, are subject to
% greater change since they have finer details that can be skipped with a
% smaller D-value. For instance, if a column alternates color every pixel,
% and you only sample every other pixel, then you will completely skip a
% color in the column. However, if the column alternates every 4 pixels and
% you sample every two, then you will retain more of the pattern that was
% in the original image.


% ------------------------------------------------------------------
% 2(e)
% ------------------------------------------------------------------
% Only need to modify function -- this area can be empty
```

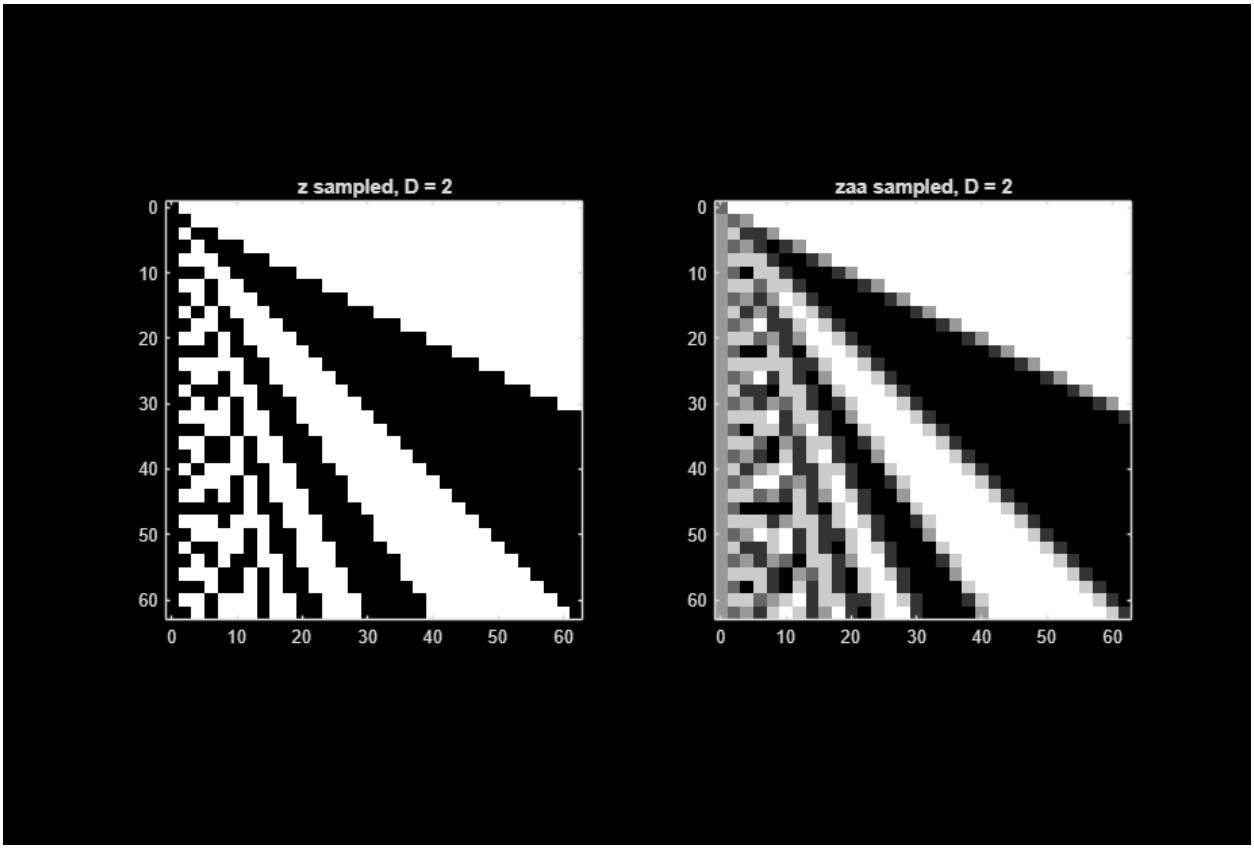
```
% ------------------------------------------------------------------
% 2(f)
% ------------------------------------------------------------------

zaa = antialias(z);    % create anti-aliased image

figure;
subplot(1,2,1);
imagesc(x, y, z);
colormap(gray);
axis image;
title('Original');

subplot(1,2,2);
imagesc(x, y, zaa);
colormap(gray);
axis image;
title('Anti-Aliased');


% ------------------------------------------------------------------
% 2(g)
% ------------------------------------------------------------------
[xs2, ys2, zs2]   = sample(z, 2);
[xs2a, ys2a, zsa2] = sample(zaa, 2);

figure;
subplot(1,2,1);
imagesc(xs2, ys2, zs2);
colormap(gray); axis image;
title('z sampled, D = 2');

subplot(1,2,2);
imagesc(xs2a, ys2a, zsa2);
colormap(gray); axis image;
title('zaa sampled, D = 2');



% ------------------------------------------------------------------
% 2(h)
% ------------------------------------------------------------------

[xs4, ys4, zs4]   = sample(z, 4);
[xs4a, ys4a, zsa4] = sample(zaa, 4);

figure;
subplot(1,2,1);
imagesc(xs4, ys4, zs4);
colormap(gray); axis image;
title('z sampled, D = 4');

subplot(1,2,2);
```

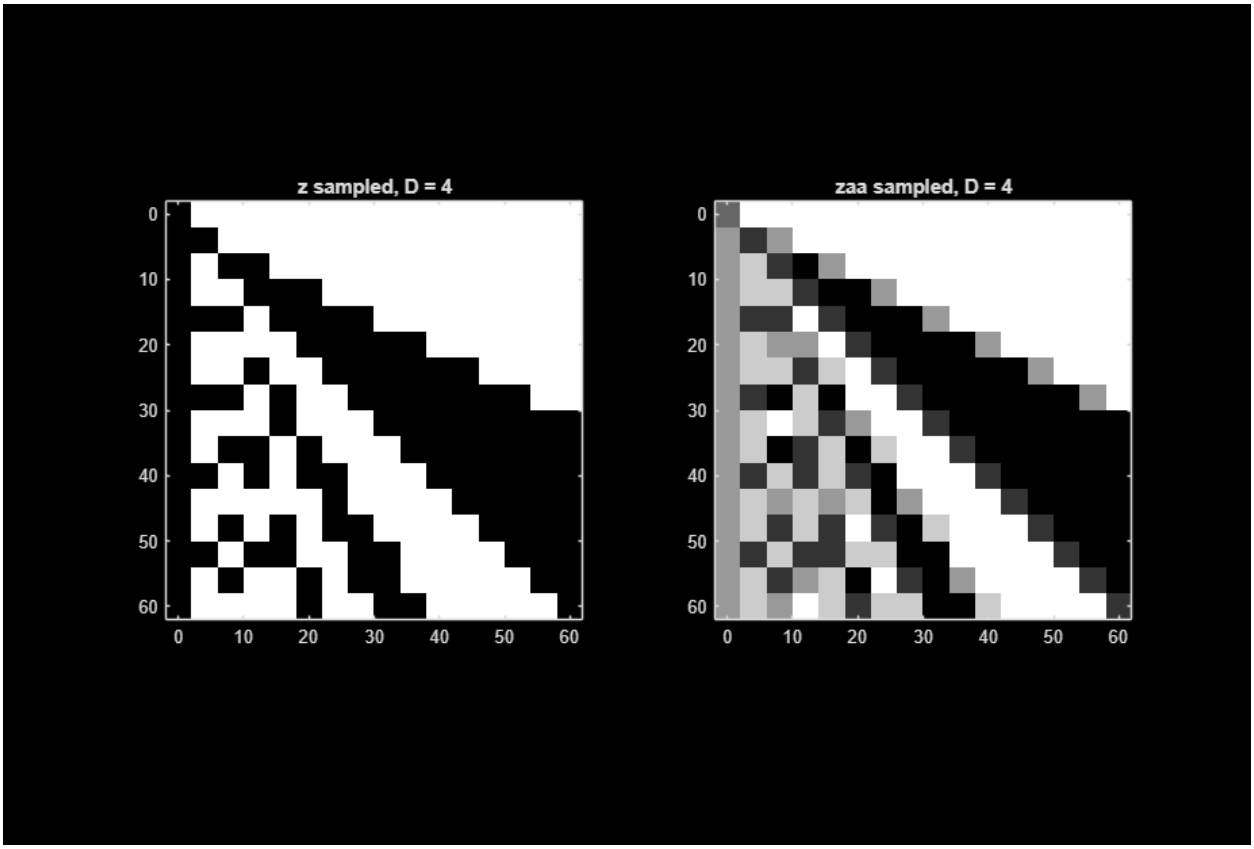
```
imagesc(xs4a, ys4a, zsa4);
colormap(gray); axis image;
title('zaa sampled, D = 4');


% ------------------------------------------------------------------
% 2(i)
% ------------------------------------------------------------------
% ANSWER QUESTION BELOW

% The anti-aliasing filter smooths the image by averaging each pixel with
its neighbors,
% which reduces sharp edges and high-frequency detail. This lowers the
amount of
% high-frequency content before sampling, so fewer distortions and false
patterns
% appear after downsampling.
```




Original Image
Sampled Image (D = 4)
0
10
20
30
40
50
60


Original
Anti-Aliased
0
10
20
30
40
50
60


z sampled, D = 2
zaa sampled, D = 2
0
10
20
30
40
50
60


z sampled, D = 4
zaa sampled, D = 4
0
10
20
30
40
50
60

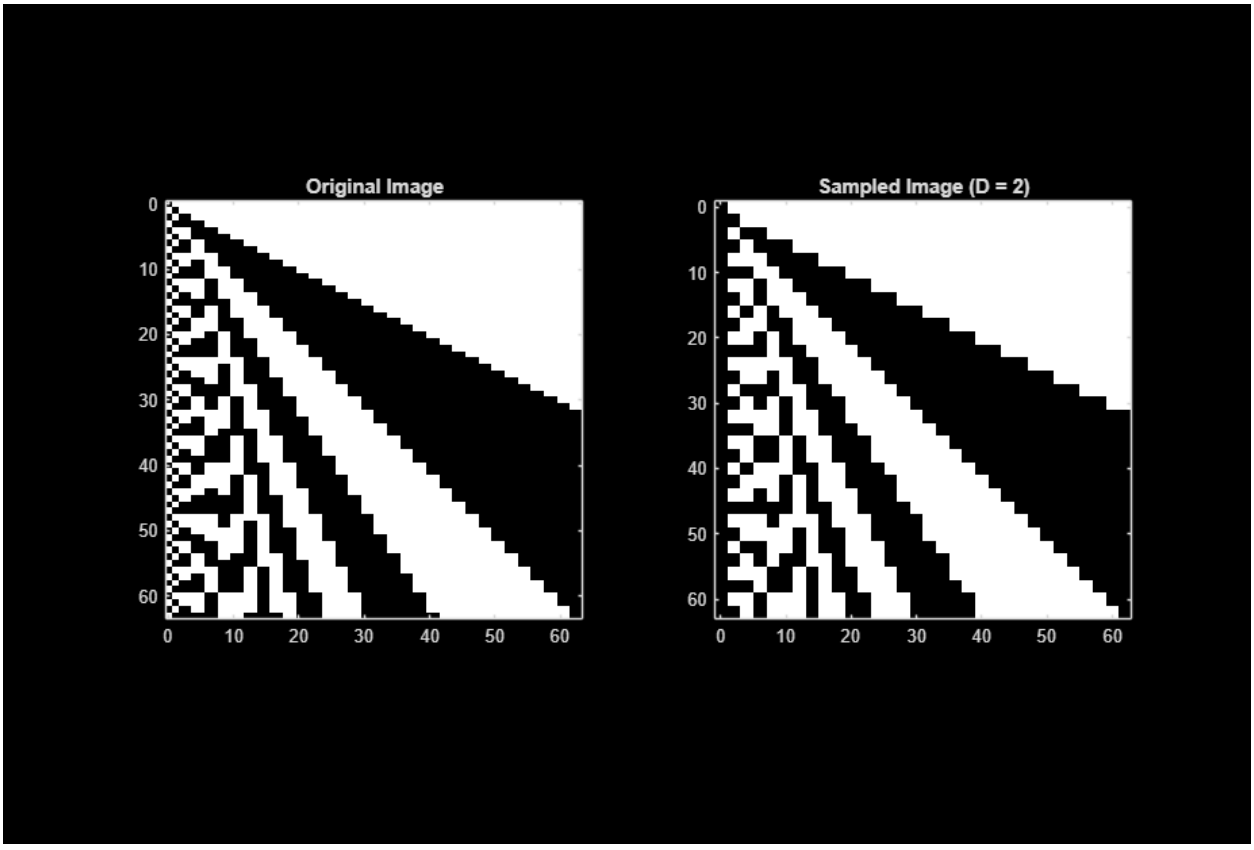
QUESTION 3 VARIABLES (DO NOT CHANGE)

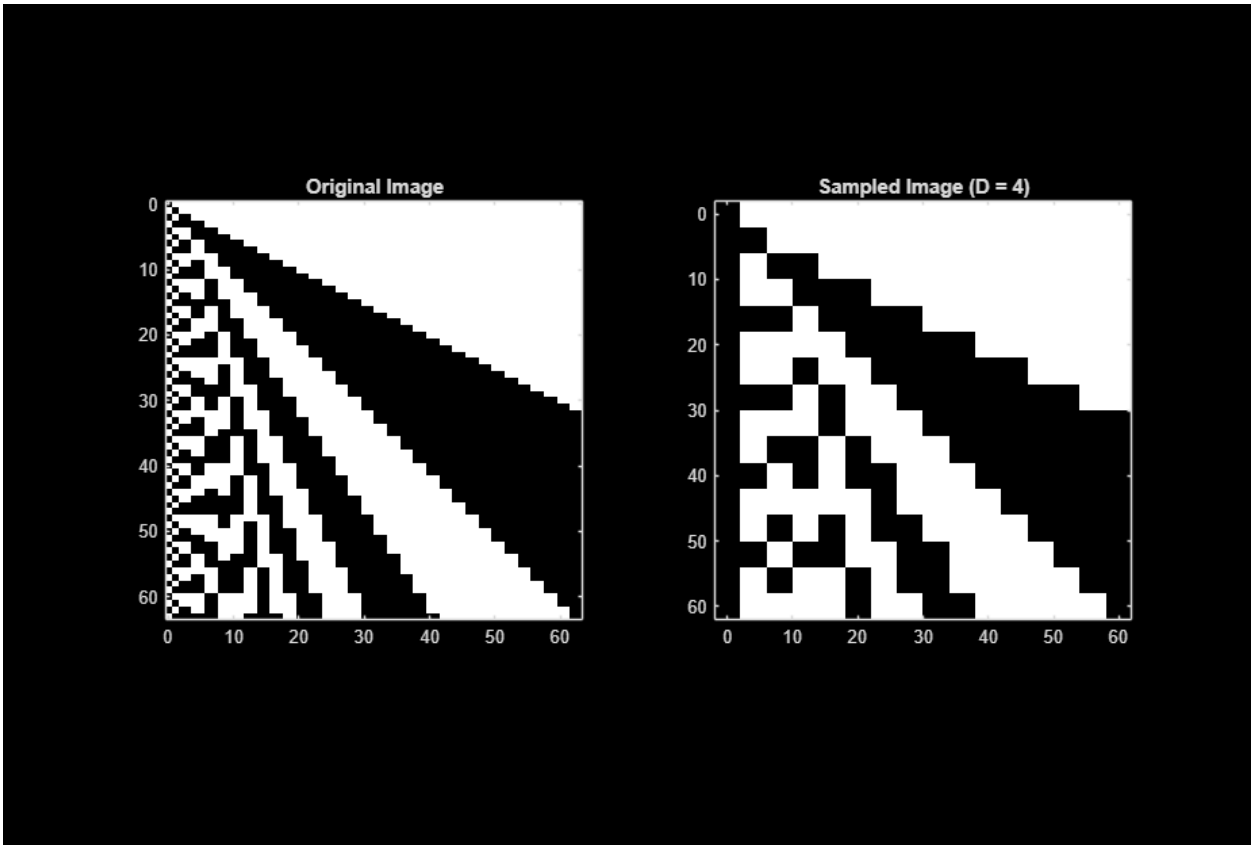
I changed 4 to 3 because I believe there was a typo.

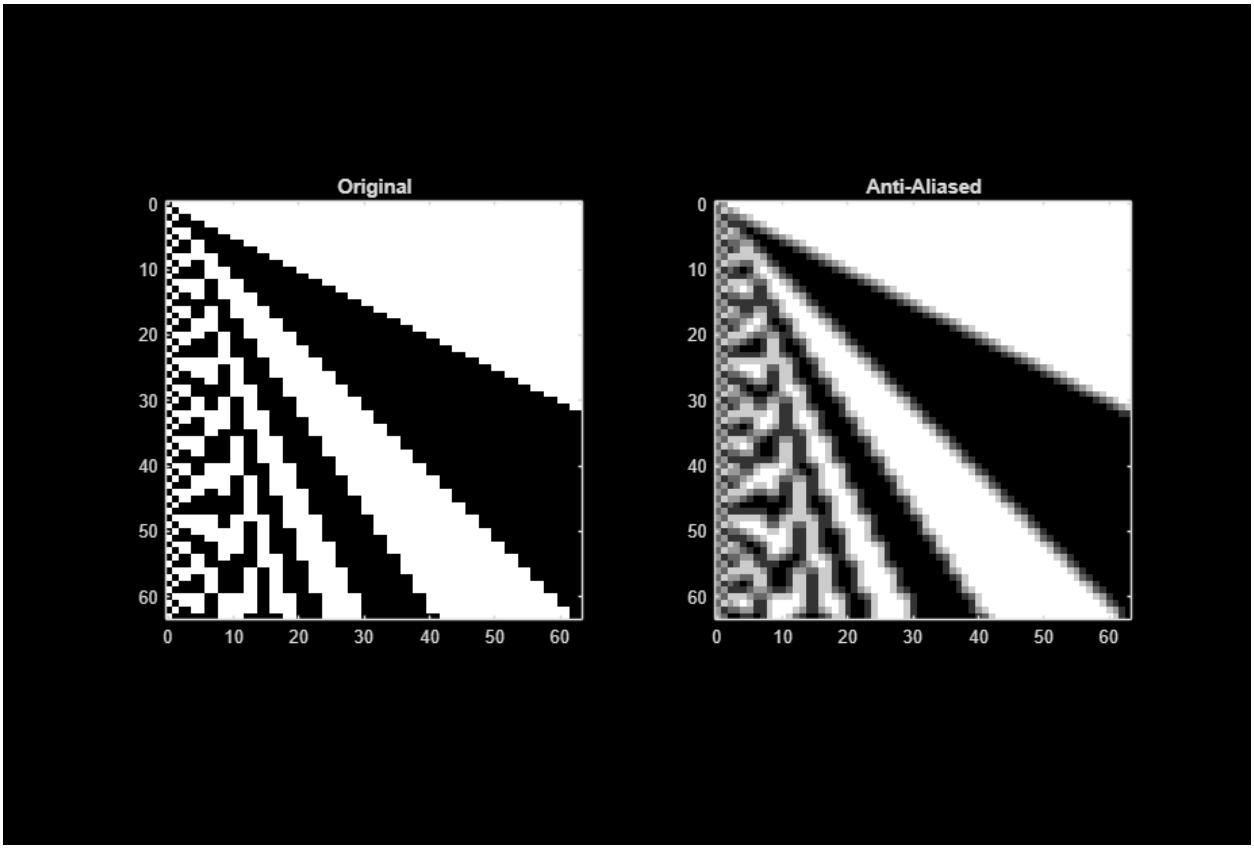
```
% LOAD VIDEO (MAKE SURE FILE BELOW IS IN THE DIRECTORY AS THIS FILE)
vid = VideoReader('wheel_video.mp4');
z = read(vid,[1 Inf]);

% The variable z is a 4 dimensional array, with dimensions 1 and 2 the
% m by n pixels in each color frame. The third dimension is the red,
% green, and blue colors in the image. The fourth dimension represents time

% HERE IS EXAMPLE CODE TO DISPLAY AND SAVE THE VIDEO AS AN MP4
% UNCOMMENT TO USE
figure;
for i = 1:min(size(z, 4),30*4)    % Only show 4 seconds max
    tic; imagesc(uint8(z(:,:,:,i))); axis square; tm = toc;
    pause(1/30-tm);               % Try to sync to 30 frames per second
end
v = VideoWriter('output_video','MPEG-4');
open(v); writeVideo(v,uint8(z)); close(v)



% ------------------------------------------------------------------
% 3(a)
% ------------------------------------------------------------------
% Only need to modify function -- this area can be empty


% ------------------------------------------------------------------
% 3(b)
% ------------------------------------------------------------------

[H,W,~,t] = size(z);

% Choose Dx and Dy so output is about 90 x 54
Dy = ceil(H/90);    % vertical step
Dx = ceil(W/54);    % horizontal step


% Dt so wheel repeats at the same phase each frame
% Observed repeat freq = 1.875 Hz -> period ≈ 0.533 s
% 30 fps * 0.533 ≈ 16 frames
Dt = 16;

% Sample the video
zs = video_sample(z, Dx, Dy, Dt);


figure;
for i = 1:min(size(zs,4), (30/Dt)*4)    % Only show 4 seconds max
    tic;
```

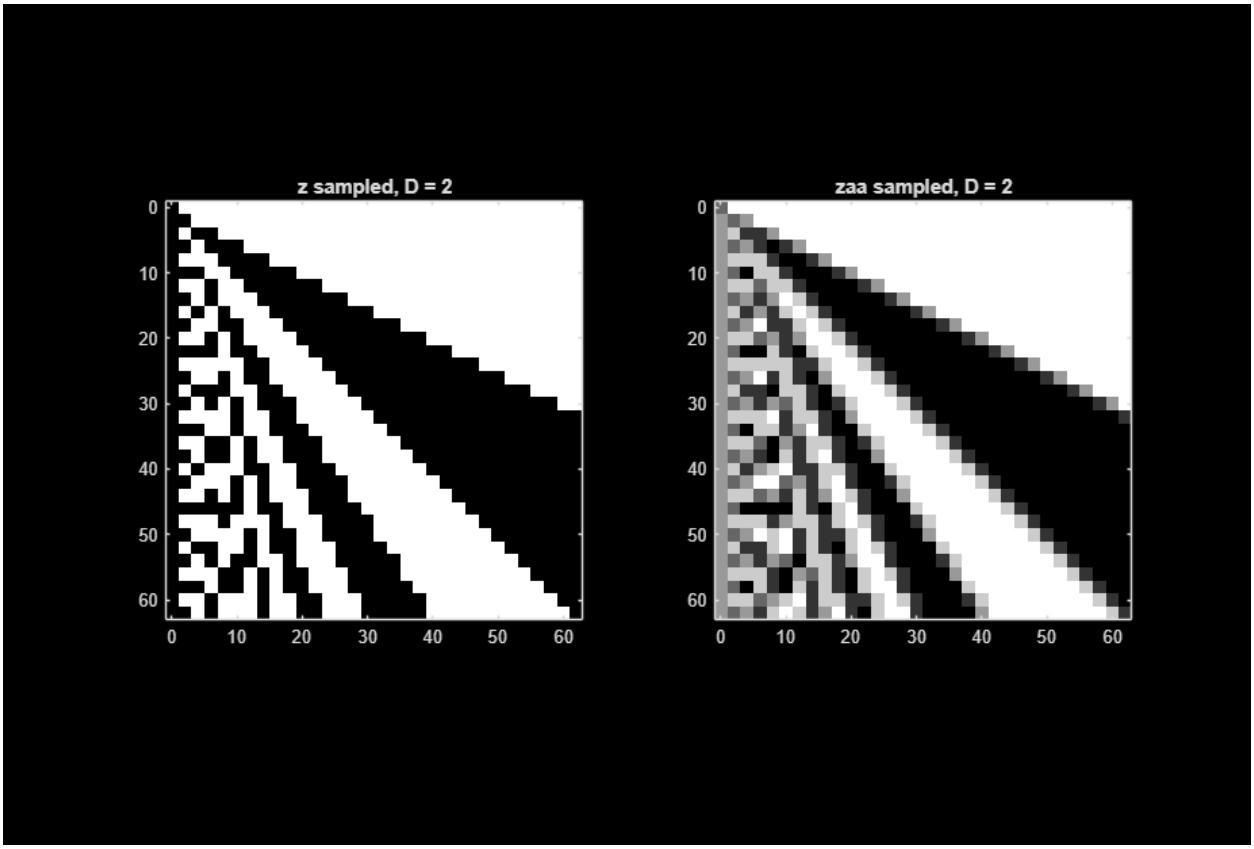
```
    imagesc(uint8(zs(:,:,:,i)));
    axis image; axis off;
    tm = toc;
    pause(1/(30/Dt) - tm);                  % Sync to sampled frame rate
end


% Save sampled video
v = VideoWriter('wheel_stuck.mp4','MPEG-4');
v.FrameRate = 30/Dt;            % keep playback speed consistent after
sampling in time
open(v);
writeVideo(v, uint8(zs));
close(v);

% ------------------------------------------------------------------
% 3(c)
% ------------------------------------------------------------------
% ANSWER QUESTION BELOW

% The wheel's is obsereved repeating wtih a rate of 1.875 Hz (0.375
rotations/s × 5 spokes),
% so the repetition period is T0 = 1/1.875 ≈ 0.533 s. At fs = 30 fps, this
is about
% 30·0.533 = around 16 frames. Choosing Dt = 16 makes it so that we sample
% the wheel around every 0.533 seconds which is when the wheel is observed
% to repeat. Each sampled frame is taken at roughly the same wheel phase
% which makes the wheel appear stationary.



% ------------------------------------------------------------------
% 3(d)
% ------------------------------------------------------------------

[H,W,~,~] = size(z);

Dy = ceil(H/90);
Dx = ceil(W/54);

Dt = 9;   % slightly off from 16 so the phase drifts backwards (aliasing)

zs_back = video_sample(z, Dx, Dy, Dt);

figure;
for i = 1:min(size(zs_back,4), (30/Dt)*4)    % Only show 4 seconds max
    tic;
    imagesc(uint8(zs_back(:,:,:,i)));
    axis image; axis off;
    tm = toc;
    pause(1/(30/Dt) - tm);                  % Sync to sampled frame rate
end

v = VideoWriter('wheel_backwards.mp4','MPEG-4');
```

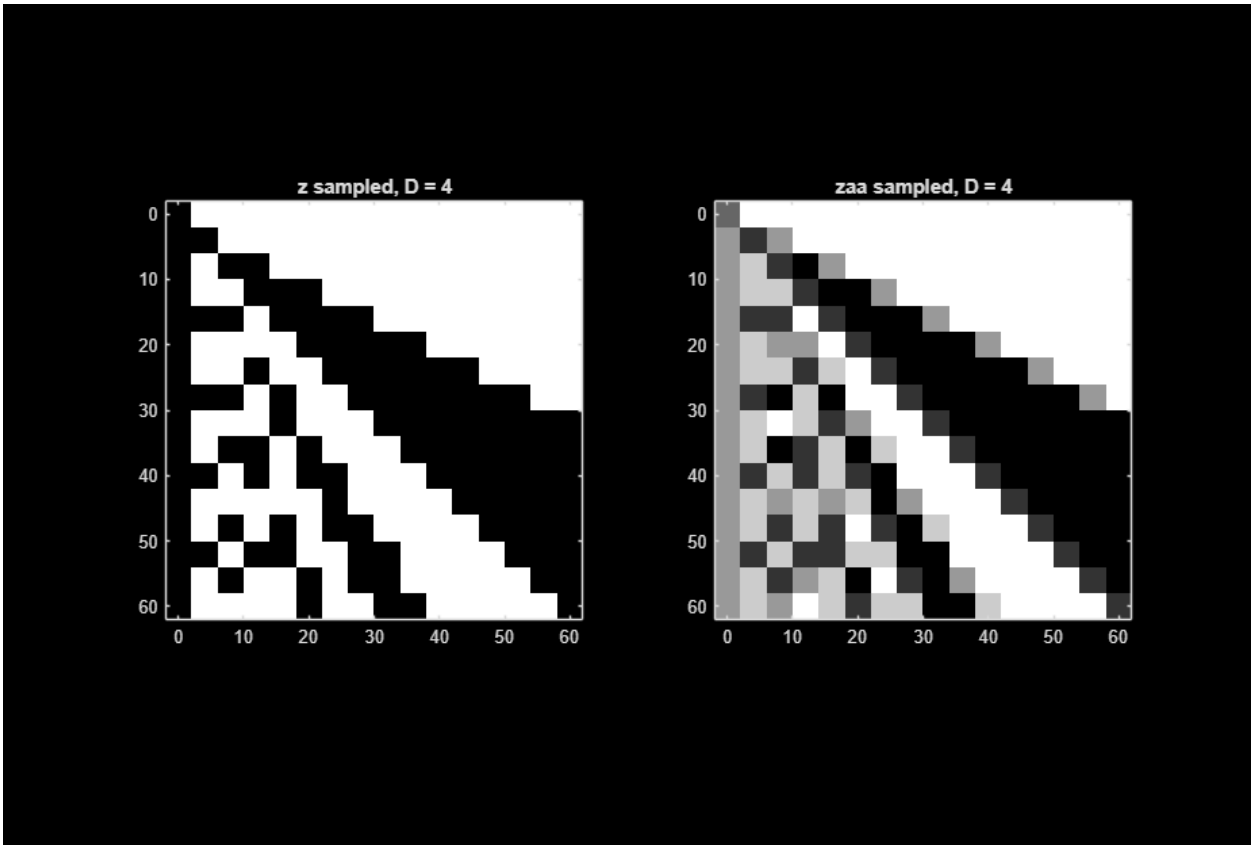
```
v.FrameRate = 30/Dt;
open(v);
writeVideo(v, uint8(zs_back));
close(v);


% ------------------------------------------------------------------
% 3(e)
% ------------------------------------------------------------------
% ANSWER QUESTION BELOW

% The wheel's observed repeating frequency is f0 ≈ 1.875 Hz. After sampling
every Dt
% frames, the effective sampling rate is fs' = 30/Dt. Backwards motion
occurs when
% fs'/2 < f0 < fs'. This gives 15/Dt < 1.875 < 30/Dt ⇒ 8 < Dt < 16.
% I set the dt equal to 9 since the wheel appears to rotate backwards at a
% decent speed. I tested other values within the range and liked the way dt
% = 9 looked the best.
```

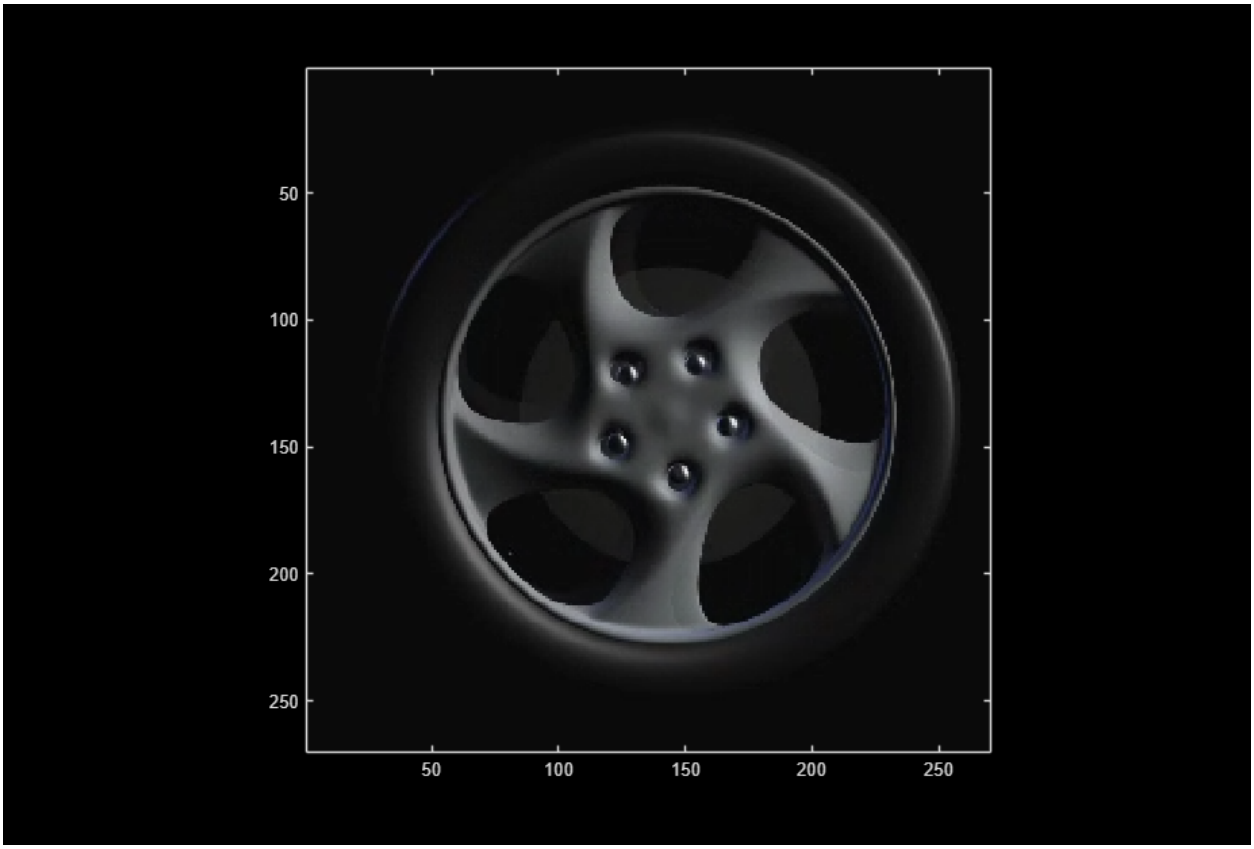



Original Image
Sampled Image (D = 4)
0
10
20
30
40
50
60


Original
Anti-Aliased
0
10
20
30
40
50
60


z sampled, D = 2
zaa sampled, D = 2
0
10
20
30
40
50
60


z sampled, D = 4
zaa sampled, D = 4
0
10
20
30
40
50
60


50
100
150
200
250
50
100
150
200
250

==========

==========

4. AI for Sampling

==========

==========

---------- 4(a) ----------

```
%I ended up chaning the value from D = 5 to D = 2 in the code manually
%after but here is the original prompt:

% You are acting as a MATLAB signal-processing assistant.
% Modify the magic_image MATLAB function shown below to combine two grayscale
% images img1 and img2 of equal size into a single output image using
% two-dimensional sampling techniques.
%
% The goal of this function is to produce an output image that appears
% visually similar to img1 when viewed at full resolution, while revealing
% img2 when the image is subsampled.
%
```

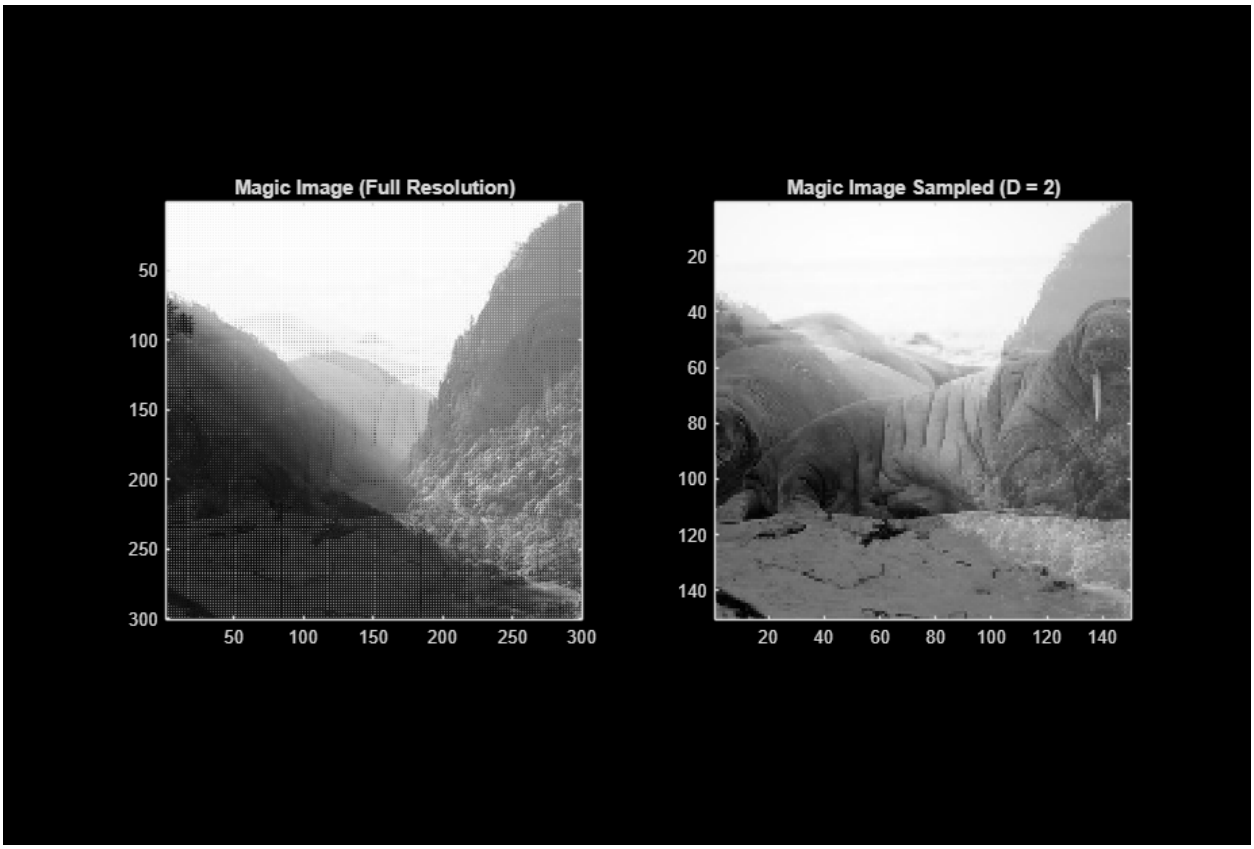
```
% The signal must satisfy all of the following properties:
% 1) The output image should appear visually similar to img1 when displayed
%    normally at full resolution.
% 2) When uniformly subsampled by selecting every 5th sample in both the
%    horizontal and vertical directions, the resulting image should appear
%    visually similar to img2.
% 3) The spatial dimensions of the output image must be the same as the
%    input images img1 and img2.
%    The subsampling pattern must be uniform (using indices 1:5:end) and
%    applied equally in both spatial directions.
% *** Show your function here ***



% ------------------------------------------------------------------
% 4(b)
% ------------------------------------------------------------------

img1 = imread('img1.jpg');
img2 = imread('img2.jpg');


% Create the "magic" combined image
img = magic_image(img1, img2);

% Sample it with D = 2
[xs, ys, img_s] = sample(img, 2);

% Plot side-by-side
figure;
subplot(1,2,1);
imagesc(img); colormap(gray); axis image; axis square;
title('Magic Image (Full Resolution)');

subplot(1,2,2);
imagesc(img_s); colormap(gray); axis image; axis square;
title('Magic Image Sampled (D = 2)');

snapnow;
```

ALL FUNCTIONS SUPPORTING THIS CODE % %

```
function [xs, ys, zs] = sample(z, D)
%SAMPLE   Sub-sample the image by keeping every D-th pixel in both rows and
columns.
%         Outputs the sampled image zs and updated x/y axes (xs, ys).

% ==> Add code here <==
% Keep every D-th pixel in the vertical and horizontal directions
zs = z(1:D:end, 1:D:end);
```

```
% New axes that match the sampled image size
xs = 0:D:(size(z,2)-1);
ys = 0:D:(size(z,1)-1);

end

function zaa = antialias(z)
%ANTIALIAS   Apply a simple 2D anti-aliasing filter by averaging each pixel
%            with its immediate horizontal and vertical neighbors.

zaa = zeros(size(z));

for y = 1:size(z,1)
    for x = 1:size(z,2)

        % Handle boundaries by using the center pixel when outside the image
        z_up    = z(max(y-1,1), x);
        z_down  = z(min(y+1,size(z,1)), x);
        z_left  = z(y, max(x-1,1));
        z_right = z(y, min(x+1,size(z,2)));

        % Apply the averaging equation
        zaa(y,x) = (1/5) * (z_up + z_left + z(y,x) + z_right + z_down);

    end
end

end


function zs = video_sample(z, Dx, Dy, Dt)
%VIDEO_SAMPLE   Subsample a video in space (x,y) and time (t)

% ==> Add code here <==
% Keep every Dy-th row, Dx-th column, and Dt-th frame
zs = z(1:Dy:end, 1:Dx:end, :, 1:Dt:end);

end




% I was not sure if I should put this here or under 4a:

 function img = magic_image(img1, img2)
%MAGIC_IMAGE  Combine two equal-size grayscale images.
%
%   IMG = MAGIC_IMAGE(IMG1, IMG2) returns an image OUT that looks like IMG1
%   when plotted normally, and becomes IMG2 after uniformly subsampling it
%   by a factor of 5 in both horizontal and vertical directions.
%
%   The function assumes IMG1 and IMG2 are 2-D numeric arrays of the same
%   dimension. Their values are interpreted in the same intensity range
%   (e.g. double in [0,1] or uint8 in [0,255]).
```

```
    % Verify that the operands are of equal size
    if ~isequal(size(img1), size(img2))
        error('Both images must have identical dimensions.');
    end

    % Start with img1 as the base image
    img = img1;

    % Embed img2 in the 5-pixel grid
    % The indices 1:5:end pick the subsampling lattice.
    img (1:2:end, 1:2:end) = img2(1:2:end, 1:2:end);

    % OPTIONAL: if you find the occasional "knock-on" pixels too obvious,
    % you can blend them slightly with the surrounding img1 values:
      alpha = 0.5;                  % 0 = fully img1, 1 = fully img2
      img(1:2:end, 1:2:end) = ...
          alpha*img2(1:2:end,1:2:end) + ...
          (1-alpha)*img1(1:2:end,1:2:end);

    % For the purposes of satisfying the three constraints, the simple
    % replacement strategy above is sufficient.

end
```



Published with MATLAB® R2025b